

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 1 – Constraint-Based Problem Solving
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	N Tharun Sai	Reg. No.:	192472382
Section / Batch:	CSE AI	Date:	26/08/26

Code of Conduct

I Tharun Sai (192472382) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tharun Sai

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Discourse, Reference Resolution & Text Coherence	Apply constraint-based problem solving for reference resolution, identify agreement and coherence constraints, resolve coreferences, and ensure entity chain consistency in a given paragraph.	Q1
Dialog and Conversational Agents, Discourse Planning & Surface Realizations	Construct constrained dialog responses by applying dialog acts, discourse planning, coherence and politeness constraints, generate multiple valid responses, and evaluate the best output.	Q2
Machine Translation, Discourse Structure, Predicate Logic & Interpretation	Perform word sense disambiguation using constraints, construct predicate logic representations, maintain discourse structure (rhetorical relations), and generate coherent translations/paraphrases.	Q3

Experiment 1: Resume Information Extraction Using Regular Expressions

Question 1:

Write a Python program using Regular Expressions to extract candidate information (name, email, mobile number, skills, and experience) from a resume, and determine shortlisting eligibility (experience ≥ 2 years and Python listed as a skill).

Aim:

To develop a Python program using Regular Expressions to extract candidate information from a resume.

Algorithm:

- Import the re module.
- Store the resume as a string.
- Extract name, email, mobile number, skills, and experience using Regex.
- Display the extracted information.
- Check eligibility (Experience ≥ 2 years and Python skill).
- Display the result.

Program (Code):

```
import re

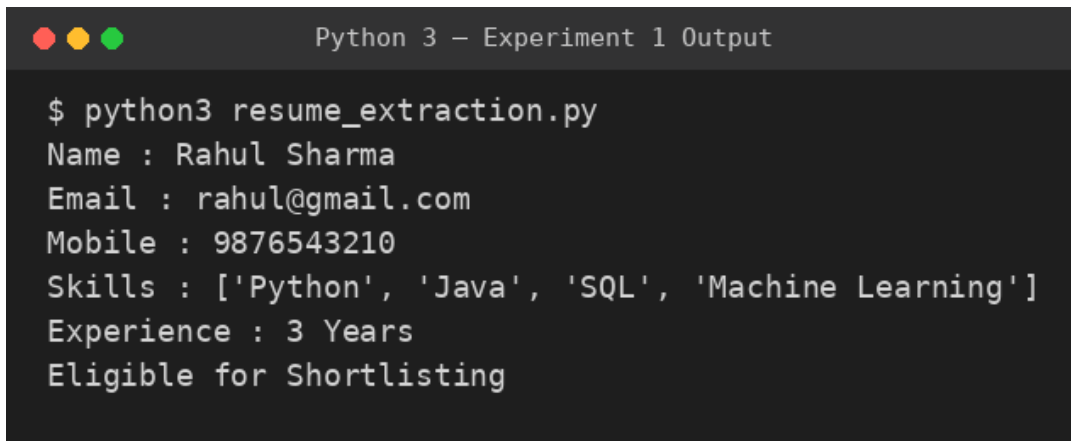
resume = """
Name: Rahul Sharma
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning
Experience: 3 years
"""

name = re.search(r"Name:\s*(.*)", resume)
email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
mobile = re.search(r"[6-9]\d{9}", resume)
skills = re.findall(r"Python|Java|SQL|Machine Learning|NLP", resume)
experience = re.search(r"(\d+)\s+years", resume)

print("Name :", name.group(1))
print("Email :", email.group())
print("Mobile :", mobile.group())
print("Skills :", skills)
print("Experience :", experience.group(1), "Years")

if int(experience.group(1)) >= 2 and "Python" in skills:
    print("Eligible for Shortlisting")
else:
    print("Not Eligible")
```

Sample Output (Screenshot):



```
Python 3 - Experiment 1 Output

$ python3 resume_extraction.py
Name : Rahul Sharma
Email : rahul@gmail.com
Mobile : 9876543210
Skills : ['Python', 'Java', 'SQL', 'Machine Learning']
Experience : 3 Years
Eligible for Shortlisting
```

Result:

Thus, the resume information was extracted successfully using Regular Expressions.

Experiment 2: Product Search System Using Regular Expressions

Question 2:

Write a Python program that searches a product list using Regular Expressions, supporting exact, prefix, suffix, partial, and case-insensitive matching, and reports the total number of matches.

Aim:

To develop a Python program for searching products using Regular Expressions.

Algorithm:

- Import the re module.
- Store product names in a list.
- Get the search keyword.
- Perform exact, prefix, suffix, partial, and case-insensitive search.
- Display matching products.
- Display total number of matches.

Program (Code):

```
import re

products = [
    "Laptop",
    "Laptop Bag",
    "Gaming Laptop",
    "Mouse",
    "Wireless Mouse",
    "Keyboard",
    "Smartphone"
]

key = input("Enter Product Keyword: ")

count = 0

print("\nMatching Products:")

for product in products:
    if re.search(key, product, re.IGNORECASE):
        print(product)
        count += 1

print("\nExact Match")
```

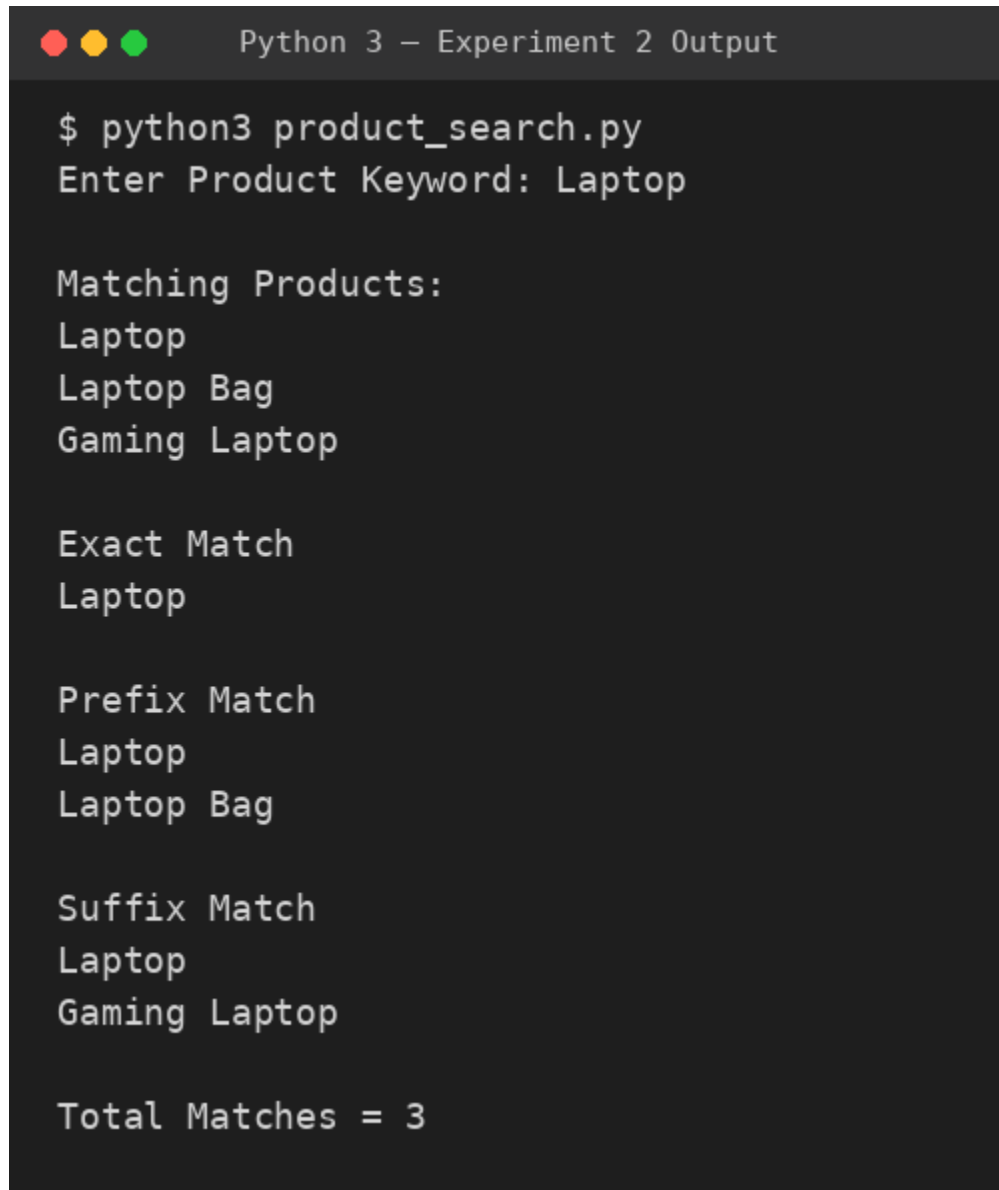
```
for product in products:
    if re.fullmatch(key, product, re.IGNORECASE):
        print(product)

print("\nPrefix Match")
for product in products:
    if re.match(key, product, re.IGNORECASE):
        print(product)

print("\nSuffix Match")
for product in products:
    if re.search(key + "$", product, re.IGNORECASE):
        print(product)

print("\nTotal Matches =", count)
```

Sample Output (Screenshot):



```
Python 3 - Experiment 2 Output

$ python3 product_search.py
Enter Product Keyword: Laptop

Matching Products:
Laptop
Laptop Bag
Gaming Laptop

Exact Match
Laptop

Prefix Match
Laptop
Laptop Bag

Suffix Match
Laptop
Gaming Laptop

Total Matches = 3
```

Result:

Thus, product searching was successfully implemented using Regular Expressions.

Experiment 3: University Registration Validation Using Regular Expressions

Question 3:

Write a Python program that validates student registration details — register number, institutional email, course code, semester, and mobile number — using Regular Expressions, and reports whether registration is successful.

Aim:

To validate student registration details using Regular Expressions.

Algorithm:

- Import the re module.
- Read student details.
- Validate Register Number.
- Validate Email ID.
- Validate Course Code.
- Validate Semester.
- Validate Mobile Number.
- Display validation status.
- Display registration result.

Program (Code):

```
import re

reg = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
sem = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

if re.fullmatch(r"\d{2}[A-Z]{3}\d{4}", reg):
    print("Register Number : Valid")
else:
    print("Register Number : Invalid")

if re.fullmatch(r"[A-Za-z0-9._%+~]+@saveetha\.com", email):
    print("Email : Valid")
else:
    print("Email : Invalid")

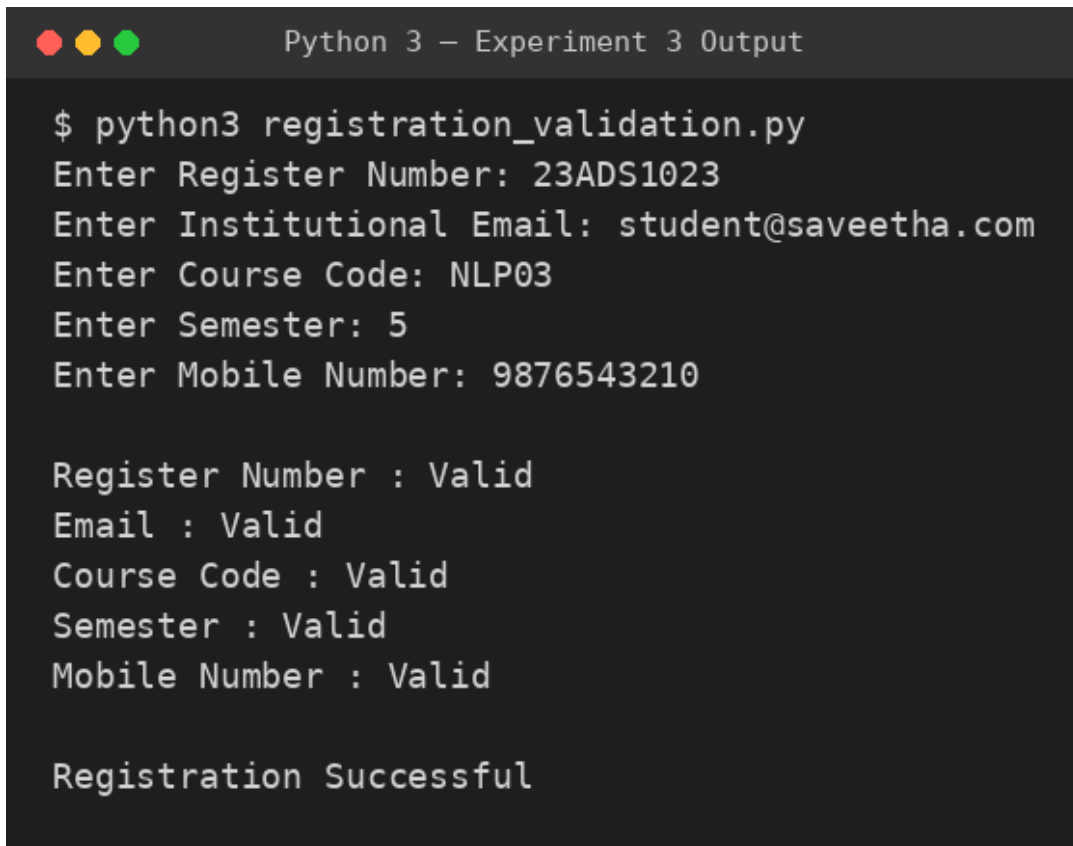
if re.fullmatch(r"[A-Z]{3}\d{2}", course):
    print("Course Code : Valid")
else:
    print("Course Code : Invalid")

if re.fullmatch(r"[1-8]", sem):
    print("Semester : Valid")
else:
    print("Semester : Invalid")

if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number : Valid")
else:
    print("Mobile Number : Invalid")

if (re.fullmatch(r"\d{2}[A-Z]{3}\d{4}", reg) and
    re.fullmatch(r"[A-Za-z0-9._%+~]+@saveetha\.com", email) and
    re.fullmatch(r"[A-Z]{3}\d{2}", course) and
    re.fullmatch(r"[1-8]", sem) and
    re.fullmatch(r"[6-9]\d{9}", mobile)):
    print("\nRegistration Successful")
else:
    print("\nRegistration Failed")
```

Sample Output (Screenshot):



```
$ python3 registration_validation.py
Enter Register Number: 23ADS1023
Enter Institutional Email: student@saveetha.com
Enter Course Code: NLP03
Enter Semester: 5
Enter Mobile Number: 9876543210

Register Number : Valid
Email : Valid
Course Code : Valid
Semester : Valid
Mobile Number : Valid

Registration Successful
```

Result:

Thus, the student registration details were successfully validated using Regular Expressions.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q. No. / Criteria Level	Problem Understanding	Constraint Analysis	Solution Development	Application of Concepts	Justification	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____